Rapport Projet SFML

Super Mario Bros

Boutigny Guillaume – Master 1 MTI3D

ENSAM – Institut de Laval

Sommaire

Introduction....3
Architecture....4
Choix de Game Design....6
IA Générative....6
Conclusion et pistes d’amélioration....7

Introduction

Dans le cadre du cours de Programmation, l'objectif du projet de fin de module était de créer ou de recréer un jeu de type platformer 2D avec divers ennemis, pièges et systèmes de gestion.

Le but était de développer ce jeu avec la librairie graphique SFML en C++ et sans autres librairies.

J'ai fais le choix de redévelopper, à ma façon, le célebre jeu Super Mario Bros original de Nintendo pour la console NES.

Dans le cadre de mes études en informatique passés, j'ai eu la chance de travailler avec beaucoup de languages de programmation, des plus simples aux plus complexes (Python, Java, Web, C#, JS, C, Java Bytecode, …). Cela a évidement un impact sur la qualité du code et c'est pour cela que je me suis le plus concentré sur l'architecture du jeu plutôt que sur le jeu en tant que tel.

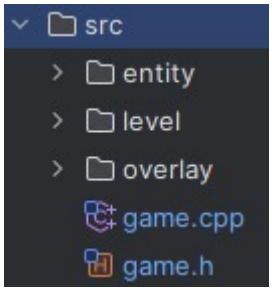
J'ai pu, de ce fait, aider mes camarades en leur fournissant une architecture de base afin de leur faciliter le développement et qu'ils puissent se concentrer purement sur le code, l'algorithmie et la logique derrière leur jeu.

Mario était à la fois un choix de coeur et de facilité du fait que tous les asssets sont disponibles librement sur le web (https://www.spriters-resource.com/nes/supermariobros/).

Architecture

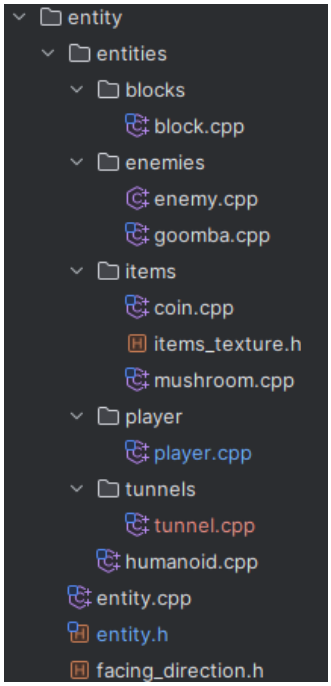
Un choix structurel a été fait de découper le projet en sous-systèmes communiquant entre eux et non pas de créer un bloc de code monolithique et inmaintainable.

Directement dans la structure des sources, on peut retrouver ces 3 systèmes :



Les **entités** sont tous les sprites qui ont une influence dans le gameplay :
- Joueur
- Pièges
- Monstres
- Blocks…

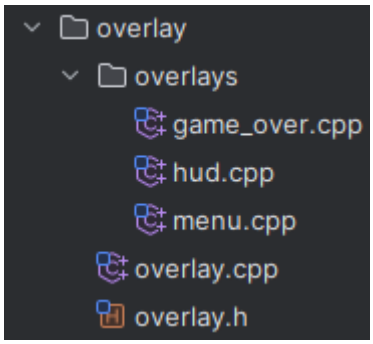
Ces entités peuvent se mouvoir ou non, être contrôler par l'utilisateur ou encore a la capacité d'interagir avec. On retrouve même spécifiquement une méthode pour savoir lorsque deux entités rentrent en collision (interactWith).



On retrouve ensuite les **niveaux (level)**. Ce sont les classes pour lesquels leurs instances représentent l'environnement dans lesquels jouent le joueur et les autres entités. Les niveaux sont instanciés dans Game au lancement du jeu puis sélectionné par leur ID par la logique même de ces niveaux (l'ID 0 est chargé à l'initialisation).
Le premier niveau dans le jeu est enfaîte le menu principal qui permet de passer au «°vrai°» premier niveau.



Le dernier sous-système présent est celui des **overlays**. Il faut voir cela comme les menus 2D qui viennent s'afficher par dessus le niveau en cours. Ils permettent de dessiner du texte, afficher des boutons et ils se mettent à jour automatiquement grâce à leur instance garder en mémoire.



Tout ces sous-systèmes sont orchestrés par la classe Game. Cette classe gère les différentes interactions et appelles pour les updates et l'affichage des levels, entities et overlays. C'est aussi dans ces fichiers que l'on retrouve notre méthode main qui créer notre instance singleton de Game, accessible statiquement.

J'ai aussi fait le choix d'utiliser un procéder de stéganographie afin d'intégrer certains éléments des levels directement dans la texture°:
La technique est d'utiliser les poids faibles des bits de couleurs afin d'encoder les informations nécessaire afin de spawn les entités et les collisions du niveau.
L'idée était simplement de réduire la taille que prendrait des assets dans un build de jeu contraint par la faible mémoire des consoles d'époque.

Choix de Game Design

Le Game Design se base exclusivement sur les mécaniques présentent dans le jeu original. Je re liste ici celles qui ont été implémentées :

- Player : Se contrôle avec les touches Q/D pour aller de Gauche à Droite et Espace pour Sauter. Le player, Mario, peut aussi évoluer dans une forme plus grande lorsqu'il récupère un champignon.
- Goomba : Les goombas sont des ennemis mobiles. Lorsque Mario entre en collision avec eux, il meurt ou redevient petit s'il était grand. Si Mario entre en collision par le DESSUS, il tue le goomba
- Blocs : Les blocs sont des objets fixes dans un niveau qui peuvent être cassé ou activé.
  - Mystery Blocs : Ils donnent une pièce ou un champignon lorsqu'ils sont activés et se transforme en bloc incassable.
  - Bricks : Ils sont des blocs cassables.
- Tunnels : Ils permettent de transitionner d'un lieu à un autre, et particulièrement d'une zone à une autre. Les zones sont des niveaux internes différents. On peut le voir et le comprendre en regardant la texture d'un niveau.
- Zone de collision dans un seul sens : Les collisions peuvent être conditionné au sens du mouvement. (ex : des plateformes sur lesquels on peut monter par le dessous mais pas passer à travers lorsque l'on est dessus)
- Mort par chute : Si Mario sort de la map, il meurt.
- Système de vie : Lorsque Mario n'a plus de vie, le jeu est terminé (« Game Over »)
- Système de pièces : Mario ramasse des pièces, au bout de 10 pièces, il gagne une vie supplémentaire.

L'animation des drapeaux n'a pas été développé. Il a été remplacé par le fait de rentrer directement dans le château pour passer au niveau suivant.

IA Générative

Comme dit lors de ma présentation, j'ai utilisé l'Intelligence Artificielle Générative lors de ce travail dans la limite strict du code pur et dur. J'ai moi-même conceptualiser puis réalisé l'architecture car je voulais développer cette compétence précise ayant déjà de l'expérience dans l'aspect algorithmique et code.

Conclusion et pistes d’amélioration

Pour conclure, ce petit projet de programmation m’a quand même permis de remettre les mains sur le langage très intéressant qu’est C++. Il m’a permis aussi d’approfondir certaines connaissances en terme de gestion de mémoire, d’ownership de pointeurs et d’autres similitudes avec d’autres langages qui m’intéresse particulièrement comme le Rust, que j’ai commencé à apprendre lors de ce même cours.

D’un point de vue purement programmation, il n’y a pas énormément d’autres points de complexités à implémenter dans l’éventualité où l’on voudrait recréer tous les niveaux et rendre le jeu complet. Il s’agirait surtout de modifier les textures et les masques afin qu’ils correspondent aux attentes du script de génération des textures stéganographiées.

La physique n’est pas non plus «°pixel perfect°» avec celle du jeu original, non pas par manque de fonctionnalité (la gravité, l’inertie et les accélérations sont là) mais dû au paramétrage. Il faudrait s’attarder plus longtemps sur les constantes en lien avec ce sujet pour perfectionner les mouvements de Mario.